

Pipeline Dataverse - Archivematica

Manuale d'uso — Pipeline Dataverse -> Archivematica

Questo manuale documenta l'uso di due script Python che automatizzano il trasferimento di dataset da **Dataverse UNIMI** (dataverse.unimi.it) ad **Archivematica** per la conservazione a lungo termine.

Script	Funzione
<code>scarica_dataverse.py</code>	Scarica dataset e metadati da Dataverse e li organizza in pacchetti compatibili con Archivematica
<code>archivematica_ingest.py</code>	Trasferisce ed esegue l'ingest dei pacchetti in Archivematica, con approvazione automatica del transfer

Indice

1. [Panoramica del flusso](#)
2. [Prerequisiti](#)
3. [Configurazione — file .env](#)
4. [Script 1 — `scarica\_dataverse.py`](#)
5. [Script 2 — `archivematica\_ingest.py`](#)
6. [Struttura del pacchetto generato](#)
7. [File di stato e report](#)
8. [Risoluzione dei problemi comuni](#)
9. [Esempio di esecuzione completa](#)

1. Panoramica del flusso

```
dois.txt                               scarica_dataverse.py           Transfer
Source
(lista DOI)  -----> - controlla se vuoto  -----> Location
               - scarica file
(Archivematica)
               - genera metadati DC
               - genera dcat.json (opt.)
               - scarica schema.json (opt.)
archivematica_ingest.py
processingMCP.xml                      - copia
transfer                              - avvia
transfer                              - auto-approva
ingest                                - attende
UUID                                  - salva AIP
```

Il flusso tipico e':

1. Si prepara `dois.txt` con l'elenco dei DOI da archiviare.
 2. Si esegue `scarica_dataverse.py`: scarica tutte le versioni di ciascun dataset, controlla che non siano vuoti, e li organizza nella Transfer Source Location di Archivematica.
 3. Si esegue `archivematica_ingest.py`: avvia il transfer e l'ingest per ogni pacchetto non ancora processato, in modo completamente automatico.
 4. Lo stato viene salvato in file JSON per le esecuzioni successive.
-

2. Prerequisiti

Software

```
pip install requests --break-system-packages
```

Accesso a Dataverse

- Una **API key Dataverse** (necessaria solo per dataset ad accesso ristretto; per dataset pubblici può essere lasciata vuota).

Accesso ad Archivematica

- **Dashboard API key** dell'utente Archivematica (es. zfed)
- **Storage Service API key** dell'utente Storage Service (es. archivematica)
- **UUID della Transfer Source Location**
- Accesso in lettura/scrittura alla Transfer Source Location sul filesystem

Permessi filesystem

```
sudo usermod -aG archivematica <utente>  
# Richiede logout/login per avere effetto
```

Note su SSL

Se Archivematica usa un certificato self-signed, impostare `SSL_VERIFY=false` nel `.env` o usare `--no-verify-ssl` da riga di comando.

URL tipici per le API: - Dashboard: `https://<host>` (porta 443) -
Storage Service: `https://<host>:8000`

3. Configurazione — file `.env`

Tutti i parametri vanno nel file `.env` nella stessa cartella degli script. Il file viene caricato automaticamente all'avvio senza dipendenze esterne.

```
cp .env.template .env  
nano .env
```

Contenuto completo del `.env`

```
# — Dataverse —  
# API key (lasciare vuoto per dataset pubblici)  
DATAVERSE_API_KEY=  
  
# Cartella di destinazione (Transfer Source Location di  
Archivematica)  
# Recuperabile con: python3 archivematica_ingest.py --list-sources  
OUTPUT_DIR=/mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE
```

```

# Esportazione DCAT-AP (dcat.json) e Schema.org (schema.json)
# per ogni versione del dataset. Equivalente al flag --dcat da CLI.
EXPORT_DCAT=false

# — Archivematica Dashboard —————
AM_URL=https://192.168.139.39
AM_USER=zfed
AM_API_KEY=

# — Archivematica Storage Service —————
SS_URL=https://192.168.139.39:8000
SS_USER=archivematica
SS_API_KEY=

# — Transfer —————
AM_TRANSFER_SOURCE_UUID=
AM_TRANSFER_TYPE=standard
AM_PROCESSING_CONFIG=dataverse_001
AM_AUTO_APPROVE=true

# — SSL —————
SSL_VERIFY=false

# — Logging —————
# Cartella dove salvare i file di log (default: logs/)
LOG_DIR=logs

```

Recuperare i valori mancanti

```

# UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Processing configuration disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-processing-
configs

```

Logging

Ogni esecuzione crea automaticamente un file di log nella cartella LOG_DIR (default: logs/):

```

logs/
  scarica_dataverse_20260619_143022.log
  archivematica_ingest_20260619_150000.log

```

Il contenuto e' identico all'output a schermo, con timestamp per ogni riga:

```

2026-06-19 14:30:22
=====
2026-06-19 14:30:22    Dataverse UNIMI – Download automatico (tutte
le versioni)
2026-06-19 14:30:23    [1/3] DOI: doi:10.13130/RD_UNIMI/KS2PUX
2026-06-19 14:30:24    --> Recupero lista versioni...

```

Per non committare i log su git:

```
echo "logs/" >> .gitignore
```

Priorita' delle variabili

Priorita'	Fonte
1 (massima)	Argomento da riga di comando
2	Variabile d'ambiente di sistema (export)
3	File .env

Attenzione: se in una sessione precedente hai eseguito `export $(grep -v '^#' .env | xargs)`, le variabili rimangono attive e hanno priorit  sul `.env`. In caso di comportamenti inattesi, aprire una nuova sessione shell.

Sicurezza

```
echo ".env" >> .gitignore
```

Errori comuni nel .env

Errore	Sintomo	Correzione
SS_URL=https://...	MissingSchema	SS_URL=https://...
SS_URL=http://...	302 Found	SS_URL=https://...
Chiavi API vuote	401 Unauthorized	Compilare le chiavi
UUID errato	404 Not Found	Usare --list-sources
Variabili esportate in sessione precedente	Valori vecchi	Nuova sessione shell

4. Script 1 — scarica_dataverse.py

Cosa fa

Per ogni DOI elencato in `dois.txt`:

1. Recupera l'elenco di tutte le versioni pubblicate
2. **Controlla se il dataset contiene file:** se e' vuoto, non crea la directory e registra il DOI in `doi_vuoti.json`
3. Per ciascuna versione:
 - scarica tutti i file del dataset
 - salva i metadati Dataverse in `dataverse.json`
 - genera `metadata.csv` Dublin Core per quella versione
 - genera `dcat.json` (DCAT-AP JSON-LD) se `EXPORT_DCAT=true`
 - scarica `schema.json` (Schema.org JSON-LD) se `EXPORT_DCAT=true`
4. Genera un `metadata.csv` complessivo nella radice del pacchetto (letto da Archivematica per il METS `dmdSec`)

Controllo dataset vuoti

Prima di creare qualsiasi directory, lo script verifica che il dataset contenga almeno un file. Se tutte le versioni sono prive di file:

- la directory del DOI **non viene creata**
- il DOI viene registrato in `doi_vuoti.json` con data e motivo
- il riepilogo mostra Vuoti: N nel conteggio finale

Cause tipiche di dataset vuoto: - dataset ancora in bozza (non ancora pubblicato) - accesso ristretto (embargo attivo) - dataset pubblicato ma senza file allegati

Esportazione DCAT-AP e Schema.org

Quando `EXPORT_DCAT=true` (nel `.env`) o `--dcat` (da CLI), per ogni versione vengono generati due file aggiuntivi:

dcat.json — DCAT-AP 2.x JSON-LD generato **localmente** dai metadati gia' scaricati, senza chiamate HTTP aggiuntive. Mapping principale:

Campo DCAT-AP	Sorgente Dataverse
---------------	--------------------

dct:title	title
dct:creator	author (nome + affiliazione)
dcat:contactPoint	datasetContact (vCard)
dct:description	dsDescription
dcat:theme	subject
dcat:keyword	keyword
dct:issued	productionDate / distributionDate
dct:publisher	publisher
dct:license	license / termsOfUse
dcat:distribution	file (formato, dimensione, checksum MD5)
owl:versionInfo	numero versione (es. 1.0)

schema.json — Schema.org JSON-LD scaricato dall'API Dataverse tramite l'exporter schema.org.

Opzioni da riga di comando

Opzione	Default	Descrizione
--dois <file>	dois.txt	File con l'elenco dei DOI
--output <cartella>	dal .env (OUTPUT_DIR)	Cartella di destinazione
--apikey <chiave>	dal .env	API key Dataverse
--no-verify-ssl	false	Disabilita verifica SSL
--dcat	false	Abilita esportazione DCAT-AP e Schema.org

Il log dell'esecuzione viene salvato automaticamente in LOG_DIR (configurabile nel .env).

Esempi

```
# Standard (tutto da .env)
python3 scarica_dataverse.py

# Con DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# Override cartella di output
python3 scarica_dataverse.py \
  --output /mnt/e/DROBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE

# Con certificato self-signed
python3 scarica_dataverse.py --no-verify-ssl
```

Output a schermo

```
=====
Dataverse UNIMI -- Download automatico (tutte le versioni)
Sorgente DOI : dois.txt (3 DOI)
Destinazione : /mnt/.../TRANSFER_SOURCE
API key      : *****5678
SSL verify   : False
Export DCAT  : True
=====

[1/3] DOI: doi:10.13130/RD_UNIMI/KS2PUX
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
```

```

-- Versione v1.0 [RELEASED]
[metadata] dataverse.json salvato
[metadata] dcat.json generato
[metadata] schema.json salvato
[data] 4 file trovati, inizio download...
    Scarico: works.tab ... OK (25 KB)
[metadata] metadata.csv (versione) generato (1 righe)
[metadata] metadata.csv generato (1 righe, 1 versione/i)
[validazione] metadata.csv OK
OK Stato: ok | Versioni: 1 | File OK: 4 | Errori: 0

[2/3] DOI: doi:10.13130/RD_UNIMI/VUOTO
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
[INFO] Dataset vuoto: 1 versione/i trovata/e ma nessun file
presente
[INFO] Directory non creata.
0 Stato: empty | Versioni: 0 | File OK: 0 | Errori: 0
Motivo: 1 versione/i trovata/e ma nessun file presente

=====
RIEPILOGO FINALE
Completati   : 2
Parziali    : 0
Errori       : 0
Vuoti        : 1
Riepilogo   : .../riepilogo_download.json
DOI vuoti    : .../doi_vuoti.json
=====

```

Comportamento idempotente

- File già scaricati: **saltati**
- dataverse.json, dcat.json, schema.json, metadata.csv di versione: generati solo se mancanti
- metadata.csv complessivo: **sempre rigenerato**
- doi_vuoti.json: **aggiornato incrementalmente** (le voci precedenti vengono preservate, non sovrascritte)

5. Script 2 — archivematica_ingest.py

Cosa fa

Per ogni pacchetto DOI nella Transfer Source Location non ancora completato:

1. Copia processingMCP.xml nella radice del pacchetto
2. Avvia il **Transfer** con nome AIP-<YYYYMMDD>-<DOI>
3. Approva automaticamente il transfer tramite API
(/api/transfer/unapproved/
 ◦ /api/transfer/approve/)
4. Attende il completamento del Transfer (polling)
5. Attende il completamento dell'**Ingest** (SIP -> AIP)
6. Salva l'UUID dell'AIP nel file di stato

Ogni cartella DOI viene trasferita come **un unico pacchetto** con tutte le sue versioni.

Opzioni da riga di comando

Sorgente e stato

Opzione	Default	Descrizione
		Directory con i

```
--source <cartella> da Transfer Source      pacchetti
--state-file <file> stato_archivematica.json File di tracciamento
```

Dashboard Archivematica

Opzione	Default	Descrizione
--am-url <url>	dal .env	URL Dashboard
--am-user <utente>	dal .env	Utente Dashboard
--am-apikey <chiave>	dal .env	API key Dashboard

Storage Service

Opzione	Default	Descrizione
--ss-url <url>	dal .env	URL Storage Service
--ss-user <utente>	dal .env	Utente Storage Service
--ss-apikey <chiave>	dal .env	API key Storage Service

Transfer

Opzione	Default	Descrizione
--transfer-source <UUID>	dal .env	UUID Transfer Source Location
--transfer-type <tipo>	standard	Tipo di transfer
--processing-config <nome>	dataverse_001	Processing configuration
--auto-approve	dal .env	Approva automaticamente il transfer

Utility

Opzione	Descrizione
--list-sources	Elenca Transfer Source Locations ed esce
--list-processing-configs	Elenca processing configuration ed esce
--no-verify-ssl	Disabilita verifica SSL
--poll-interval <sec>	Intervallo polling (default: 15s)
--dry-run	Mostra cosa farebbe senza eseguire

Il log dell'esecuzione viene salvato automaticamente in LOG_DIR (configurabile nel .env).

Esempi

```
# Standard (tutto da .env)
python3 archivematica_ingest.py

# Lista Transfer Source disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Anteprima senza eseguire
python3 archivematica_ingest.py --dry-run

# Processing configuration diversa
python3 archivematica_ingest.py --processing-config automated
```

Output a schermo

```
=====
Archivematica Ingest -- trasferimento automatico dataset
Sorgente      : /mnt/.../TRANSFER_SOURCE
Pacchetti DOI : 3
Dashboard     : https://192.168.139.39 (utente: zfed)
Storage Svc   : https://192.168.139.39:8000 (utente:
archivematica)
Transfer Src   : 76ab3067-...
Transfer type  : standard
Processing cfg : dataverse_001
Auto-approve  : True
SSL verify    : False
File di stato : stato_archivematica.json
=====

[1/3] doi_10.13130_RD_UNIMI_KS2PUX
--> Elaborazione: doi_10.13130_RD_UNIMI_KS2PUX
[debug] processingMCP.xml ('dataverse_001') copiato in
.../processingMCP.xml
Avvio transfer 'AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX' ...
Transfer UUID: <uuid>
[approve] Attendo 10s ... (tentativo 1/5)
[approve] Trovato: AIP-20260619-...-<uuid> -- invio approvazione
...
[approve] Approvato (UUID: <uuid>)
Attendo completamento transfer ...
[transfer] stato: PROCESSING -- attendo 15s ...
SIP UUID: <uuid>
Attendo completamento ingest ...
OK Ingest completato -- AIP UUID: <uuid>

=====
RIEPILOGO FINALE
Completati   : 3
Gia' presenti: 0
Errori      : 0
=====
```

6. Struttura del pacchetto generato

```
TRANSFER_SOURCE/
doi_10.13130_RD_UNIMI_KS2PUX/
objects/
  v1.0/
    objects/          <- file del dataset
    works.tab
    README.txt
    metadata/         <- metadati di questa versione
    dataverse.json    <- metadati Dataverse
  (preservation)
    metadata.csv      <- Dublin Core di questa
versione
    dcat.json         <- DCAT-AP JSON-LD (solo se
EXPORT_DCAT=true)
    schema.json       <- Schema.org JSON-LD (solo se
EXPORT_DCAT=true)
  v2.0/
    objects/
    works_v2.tab
    metadata/
    dataverse.json
    metadata.csv
    dcat.json         <- (solo se EXPORT_DCAT=true)
    schema.json       <- (solo se EXPORT_DCAT=true)
```



```

    metadata/
      metadata.csv          <- Dublin Core TUTTE le versioni
-> METS dmdSec
    processingMCP.xml      <- copiato da
archivematica_ingest.py
    riepilogo_download.json <- riepilogo scaricamenti
    doi_vuoti.json         <- DOI senza file (aggiornato
incrementalmente)

```

Formato metadata.csv (radice del pacchetto)

filename	dc.title	dc.creator	dc.date	
objects/v1.0/objects/works.tab	Titolo	Autore	2026-03-04	c
objects/v2.0/objects/works_v2.tab	Titolo	Autore	2026-03-04	c

Archivematica traspone questi metadati nel METS.xml come <dmdSec MDTYPE="DC">.

7. File di stato e report

File di log

Generati da entrambi gli script in logs/ (o nella cartella indicata da LOG_DIR):

```

logs/
  scarica_dataverse_<YYYYMMDD>_<HHMMSS>.log
  archivematica_ingest_<YYYYMMDD>_<HHMMSS>.log

```

Ogni file contiene l'intero output dell'esecuzione con timestamp per ogni riga. I log vengono accumulati — ogni esecuzione crea un nuovo file senza sovrascrivere quelli precedenti.

riepilogo_download.json

Generato da scarica_dataverse.py. Riepilogo completo di ogni DOI:

```

[
  {
    "doi": "doi:10.13130/RD_UNIMI/KS2PUX",
    "status": "ok",
    "versions": [{"version": "v1.0", "files_ok": 4, "files_err":
0}],
    "error": null,
    "empty_reason": null
  },
  {
    "doi": "doi:10.13130/RD_UNIMI/VUOTO",
    "status": "empty",
    "versions": [],
    "error": null,
    "empty_reason": "1 versione/i trovata/e ma nessun file presente"
  }
]

```

doi_vuoti.json

Generato da scarica_dataverse.py. Contiene solo i DOI senza file, aggiornato incrementalmente ad ogni esecuzione:

```
[
```

```
{
  "doi": "doi:10.13130/RD_UNIMI/VUOTO",
  "empty_reason": "1 versione/i trovata/e ma nessun file presente
(dataset vuoto o accesso ristretto)",
  "rilevato_il": "2026-06-19T10:30:00+00:00"
}
```

stato_archivematica.json

Generato da archivematica_ingest.py. Traccia lo stato di ogni transfer:

```
{
  "doi_10.13130_RD_UNIMI_KS2PUX": {
    "transfer_name": "AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX",
    "transfer_uuid": "bde477ff-...",
    "sip_uuid": "a1b2c3d4-...",
    "aip_uuid": "e5f6a7b8-...",
    "status": "ingested",
    "completed_at": "2026-06-19T10:42:13+00:00",
    "error": null
  }
}
```

Valori di status (stato_archivematica.json)

Valore	Significato
in_progress	Avviato ma non completato
transferred	Transfer ok, ingest non completato
ingested	Completato – saltato nelle esecuzioni successive
failed	Errore – ritentato da zero al prossimo avvio

Reset manuale di un pacchetto

```
python3 -c "
import json
s = json.load(open('stato_archivematica.json'))
s.pop('doi_10.13130_RD_UNIMI_KS2PUX', None)
json.dump(s, open('stato_archivematica.json', 'w'), indent=2)
"
```

Reset completo

```
echo '{}' > stato_archivematica.json
```

8. Risoluzione dei problemi comuni

MissingSchema / URL non valido

MissingSchema: Invalid URL 'https://...'

Aggiungere i due punti: https:// non https//.

302 Found sulla porta 8000

```
SS_URL=https://192.168.139.39:8000
```

401 Unauthorized

Verificare AM_API_KEY e SS_API_KEY nel .env.

404 Not Found sulla Transfer Source UUID

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

Aggiornare AM_TRANSFER_SOURCE_UUID nel .env con l'UUID corretto per il server in uso (ogni installazione Archivematica ha UUID diversi).

SSL Certificate Verify Failed

```
SSL_VERIFY=false # nel .env
# oppure
python3 archivematica_ingest.py --no-verify-ssl
```

EXPORT_DCAT=true nel .env ma dcat.json non viene generato

Verificare che non ci siano variabili esportate in precedenza con export che sovrascrivono il .env:

```
unset EXPORT_DCAT
python3 scarica_dataverse.py --dcat # oppure rilancia in una nuova sessione
```

Permission Denied su processingMCPConfigs

```
sudo usermod -aG archivematica <utente>
# Poi fare logout e login
```

Transfer REJECTED al riavvio

Lo script rileva automaticamente status failed e riparte da zero.

400 "Unable to determine the status of the unit"

Errore transitorio: lo script ritenta automaticamente 2 volte (20s di attesa).

Auto-approve non funziona

1. Verificare AM_AUTO_APPROVE=true nel .env
2. Verificare che il transfer sia in USER_INPUT nel Dashboard
3. Aumentare APPROVE_TRANSFER_WAIT nel codice (default: 10s) se Archivematica e' lenta ad avviare il transfer

Elasticsearch non raggiungibile

```
sudo systemctl start elasticsearch
curl -s http://localhost:9200
```

9. Esempio di esecuzione completa

Configurazione iniziale (una tantum)

```
git clone https://github.com/zfed/Arkive.git
cd Arkive/tools/dataverse-archivematica

pip install requests --break-system-packages

cp .env.template .env
nano .env
# Compilare almeno:
# OUTPUT_DIR, DATAVERSE_API_KEY, AM_API_KEY, SS_API_KEY,
```

```
# AM_TRANSFER_SOURCE_UUID, SSL_VERIFY

# Recupera UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Verifica processing configuration
python3 archivematica_ingest.py --no-verify-ssl --list-processing-
configs
```

Esecuzione standard

```
# 1. Prepara la lista DOI
cat > dois.txt << 'EOF'
doi:10.13130/RD_UNIMI/KS2PUX
doi:10.13130/RD_UNIMI/NFURUT
doi:10.13130/RD_UNIMI/QBRJNN
EOF

# 2. Scarica i dataset (OUTPUT_DIR letto dal .env)
python3 scarica_dataverse.py

# 2b. Con metadati DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# 3. Verifica i dataset vuoti (se presenti)
cat doi_vuoti.json | python3 -m json.tool

# 4. Ingesta in Archivematica (completamente automatico)
python3 archivematica_ingest.py

# 5. Verifica lo stato
cat stato_archivematica.json | python3 -m json.tool
```

Per le esecuzioni successive, ripetere i passi 2 e 4: gli script elaborano solo cio' che e' nuovo o non ancora completato.